SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

Course Code:	DSA05	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	Tool 2 – Schema Analysis and Viva Voce
Weightage:		**Total Marks:**	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)			
Student Name:	**S.Nandini**	**Reg. No.:**	**192473030**
Section / Batch:	**Slot A/ 2024**	**Date of Submission:**	
Faculty In charge:	K. Veena Devi	**Project Mode:**	Individual Submission

Code of Conduct

I __S.Nandini(192473030)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____

Identification of Entities and Attributes (4 Marks)	Understanding of Relationships (4 Marks)	Analysis of Keys and Constraints (4 Marks)	Detection of Design Issues (4 Marks)	Viva Voce (4 Marks)	Total (20 Marks)

1. University Course Registration System

A university database contains the following tables:

- Student(StudentID, Name, DepartmentID)
- Department(DepartmentID, DepartmentName)
- Course(CourseID, CourseName, FacultyID)
- Faculty(FacultyID, FacultyName)
- Enrollment(StudentID, CourseID, Semester)

A student can enroll in multiple courses, and each course can have multiple students.

Database Schema

Student

- StudentID (Primary Key)
- Name
- DepartmentID (Foreign Key)

Department

- DepartmentID (Primary Key)
- DepartmentName

Course

- CourseID (Primary Key)
- CourseName
- FacultyID (Foreign Key)

Faculty

- FacultyID (Primary Key)
- FacultyName

Enrollment

- StudentID (Foreign Key)
- CourseID (Foreign Key)
- Semester

Relationship

- One department can have many students.
- One faculty member can teach many courses.

- One student can enroll in many courses.
- One course can have many students.
- The **Enrollment** table represents the many-to-many relationship between students and courses.

Viva Questions

Q1. Why is the Enrollment table necessary instead of storing CourseID directly in the Student table?

The Enrollment table is necessary because the relationship between students and courses is **many-to-many**.

- A student can enroll in multiple courses.
- A course can have many students.
- If CourseID were stored directly in the Student table, only one course could be assigned to each student.
- The Enrollment table avoids data redundancy and follows database normalization principles.
- It also allows storing additional information such as Semester, Grade, Attendance, etc.

Q2. If the university wants to track grades for each enrolled course, how would you modify the schema?

The Enrollment table can be modified by adding a **Grade** attribute.

Modified Enrollment Table

StudentID	CourseID	Semester	Grade
Foreign Key	Foreign Key	Semester	Grade

This allows recording the grade earned by a student for each course in a particular semester without affecting other tables.

2. Hospital Management System

A hospital database contains:

- Patient(PatientID, Name, Age)
- Doctor(DoctorID, DoctorName, Specialization)
- Appointment(AppointmentID, PatientID, DoctorID, Date)
- Prescription(PrescriptionID, AppointmentID, Medicine)

Multiple patients can consult the same doctor.

Database Schema

Patient

- PatientID (Primary Key)
- Name
- Age

Doctor

- DoctorID (Primary Key)
- DoctorName
- Specialization

Appointment

- AppointmentID (Primary Key)
- PatientID (Foreign Key)
- DoctorID (Foreign Key)
- Date

Prescription

- PrescriptionID (Primary Key)
- AppointmentID (Foreign Key)
- Medicine

Relationship

- One patient can have multiple appointments.
- One doctor can attend many patients.
- Each appointment can generate one or more prescriptions.
- The Prescription table is linked to the Appointment table.

Viva Questions

Q1. What problems might occur if prescription details are stored directly in the Appointment table?

Storing prescription details directly in the Appointment table creates several problems:

- Data redundancy occurs when multiple medicines need to be stored.
- The Appointment table becomes unnecessarily large.

- It violates database normalization.
- Updating prescription information becomes difficult.
- Multiple medicines cannot be stored efficiently in a single field.
- It may lead to data inconsistency.

Therefore, keeping prescriptions in a separate table improves flexibility and data integrity.

Q2. How would the schema change if a prescription contains multiple medicines?

If a prescription contains multiple medicines, the Prescription table can be redesigned as follows:

Prescription

- PrescriptionID (Primary Key)
- AppointmentID (Foreign Key)

PrescriptionMedicine

- PrescriptionID (Foreign Key)
- MedicineName
- Dosage
- Quantity

This design allows one prescription to contain multiple medicines while maintaining proper normalization.

3. Online Shopping System

Database tables include:

- Customer(CustomerID, Name)
- Product(ProductID, ProductName, Price)
- Order(OrderID, CustomerID, OrderDate)
- OrderDetails(OrderID, ProductID, Quantity)

Each order may contain multiple products.

Database Schema

Customer

- CustomerID (Primary Key)
- Name

Product

- ProductID (Primary Key)

- ProductName
- Price

Order

- OrderID (Primary Key)
- CustomerID (Foreign Key)
- OrderDate

OrderDetails

- OrderID (Foreign Key)
- ProductID (Foreign Key)
- Quantity

Relationship

- One customer can place many orders.
- One order can contain multiple products.
- One product can appear in many different orders.
- The OrderDetails table represents the many-to-many relationship between orders and products.

Viva Questions

Q1. Why is the OrderDetails table required in this schema?

The OrderDetails table is required because:

- One order may contain multiple products.
- One product can appear in multiple orders.
- It stores the quantity of each product purchased.
- It removes duplicate data.
- It maintains database normalization.
- It allows additional information such as discount, selling price, or tax to be stored for each ordered product.

Q2. What would happen if ProductID were stored directly in the Order table?

If ProductID were stored directly in the Order table:

- An order could contain only one product.
- Multiple products would require duplicate order records.
- Data redundancy would increase.
- Updating order information would become difficult.
- The database would violate normalization rules.

- It would not correctly represent the one-to-many relationship between an order and its products.

Therefore, the OrderDetails table is necessary to efficiently manage orders containing multiple products.

4. Library Management System
Database tables:
- Member(MemberID, Name)
- Book(BookID, Title, Author)
- Issue(MemberID, BookID, IssueDate, ReturnDate)

Members can borrow multiple books.

Database Schema

Member

- MemberID (Primary Key)
- Name

Book

- BookID (Primary Key)
- Title
- Author

Issue

- MemberID (Foreign Key)
- BookID (Foreign Key)
- IssueDate
- ReturnDate

Viva Questions

Q1. Explain the relationship between Member and Book through the Issue table.

The relationship between Member and Book is many-to-many. A member can borrow multiple books, and a book can be borrowed by different members over time. The Issue table acts as a bridge table by storing MemberID, BookID, IssueDate, and ReturnDate, allowing efficient tracking of issued books.

Q2. How would you modify the schema to track overdue fines?

A **Fine** attribute can be added to the Issue table.

Modified Issue Table

- MemberID
- BookID
- IssueDate
- ReturnDate
- Fine

This stores the overdue fine for each issued book.

---

5. Banking System
Tables:

- Customer(CustomerID, Name)
- Account(AccountNo, CustomerID, Balance)
- Transaction(TransactionID, AccountNo, Amount, Type)

One customer can have multiple accounts.

Database Schema

Customer

- CustomerID (Primary Key)
- Name

Account

- AccountNo (Primary Key)
- CustomerID (Foreign Key)
- Balance

Transaction

- TransactionID (Primary Key)
- AccountNo (Foreign Key)
- Amount
- Type

Relationship

- One customer can have multiple accounts.

- One account can have many transactions.

Viva Questions

Q1. Why should transactions be stored separately from account details?

Transactions are stored separately because an account can have many transactions over time. Keeping them in a separate table maintains a complete transaction history, reduces redundancy, improves database performance, and follows normalization principles.

Q2. How would you redesign the schema to support joint accounts?

Create a new table called **CustomerAccount**.

CustomerAccount

- CustomerID (Foreign Key)
- AccountNo (Foreign Key)

This allows multiple customers to own the same account while maintaining a normalized database.

6. Employee Payroll System

Tables:

- Employee(EmployeeID, Name, DepartmentID)
- Department(DepartmentID, DepartmentName)
- Payroll(PayrollID, EmployeeID, Salary, Month)

Employees belong to departments and receive monthly salaries.

Database Schema

Employee

- EmployeeID (Primary Key)
- Name
- DepartmentID (Foreign Key)

Department

- DepartmentID (Primary Key)
- DepartmentName

Payroll

- PayrollID (Primary Key)
- EmployeeID (Foreign Key)

- Salary
- Month

Relationship

- One department has many employees.
- One employee receives a payroll record every month.

Viva Questions

Q1. Why is Department information separated into a different table?

Department information is stored separately to eliminate duplicate data. It ensures that department details are maintained in one place, making updates easier and improving data consistency.

Q2. What redundancy issues would arise if department details were stored in Employee records?

Department information would be repeated for every employee working in the same department. This increases storage space and may lead to update, insertion, and deletion anomalies, causing inconsistent data.

7. Hotel Reservation System

Tables:

- Customer(CustomerID, Name)
- Room(RoomID, RoomType)
- Reservation(ReservationID, CustomerID, RoomID, CheckIn, CheckOut)

Rooms can be booked by different customers at different times.

Database Schema

Customer

- CustomerID (Primary Key)
- Name

Room

- RoomID (Primary Key)
- RoomType

Reservation

- ReservationID (Primary Key)
- CustomerID (Foreign Key)
- RoomID (Foreign Key)
- CheckIn
- CheckOut

Relationship

- One customer can make multiple reservations.
- One room can be booked by different customers at different times.

Viva Questions

Q1. Why should reservation details be maintained separately from room details?

Reservation details change frequently, while room details remain mostly unchanged. Storing reservations separately allows the hotel to maintain booking history, avoid redundancy, and efficiently manage room availability.

Q2. How would you prevent double booking of rooms using the schema?

Before creating a new reservation, the system should check whether another reservation already exists for the same RoomID during the requested check-in and check-out dates. If an overlapping reservation exists, the booking should be rejected.

8. Food Delivery Application

Tables:

- Customer(CustomerID, Name)
- Restaurant(RestaurantID, Name)
- Order(OrderID, CustomerID, RestaurantID)
- DeliveryAgent(AgentID, Name)
- Delivery(OrderID, AgentID)

Orders are delivered by delivery agents.

Database Schema

Customer

- CustomerID (Primary Key)

- Name

Restaurant

- RestaurantID (Primary Key)
- Name

Order

- OrderID (Primary Key)
- CustomerID (Foreign Key)
- RestaurantID (Foreign Key)

DeliveryAgent

- AgentID (Primary Key)
- Name

Delivery

- OrderID (Foreign Key)
- AgentID (Foreign Key)

Relationship

- One customer can place many orders.
- One restaurant can receive many orders.
- One delivery agent can deliver multiple orders.

Viva Questions

Q1. Why is Delivery maintained as a separate table?

The Delivery table links orders with delivery agents. Keeping delivery information separate avoids duplication, makes it easier to assign agents, and maintains a normalized database structure.

Q2. How would the schema change if multiple delivery agents could handle a single order?

The Delivery table can store multiple records with the same OrderID and different AgentIDs. Additional fields such as DeliveryStage or AssignedTime can also be added to track each agent's responsibility.

9. Learning Management System (LMS)

Tables:

- Student(StudentID, Name)
- Course(CourseID, CourseName)
- Enrollment(StudentID, CourseID)
- Assignment(AssignmentID, CourseID)

Students can enroll in many courses.

tudent

- StudentID (Primary Key)
- Name

Course

- CourseID (Primary Key)
- CourseName

Enrollment

- StudentID (Foreign Key)
- CourseID (Foreign Key)

Assignment

- AssignmentID (Primary Key)
- CourseID (Foreign Key)

Relationship

- One student can enroll in multiple courses.
- One course can have multiple students.
- One course can contain multiple assignments.

Viva Questions

Q1. Why is Enrollment considered a bridge table?

Enrollment is called a bridge table because it connects the Student and Course tables. It resolves the many-to-many relationship by storing both StudentID and CourseID.

Q2. How would you modify the schema to record assignment grades?

Create a new table called **Submission**.

Submission

- StudentID (Foreign Key)
- AssignmentID (Foreign Key)
- Grade

This table stores the grade obtained by each student for every assignment.

10. Bus Ticket Reservation System

Tables:

- Passenger(PassengerID, Name)
- Bus(BusID, Route)
- Booking(BookingID, PassengerID, BusID, TravelDate)

Passengers can make multiple bookings.

Database Schema

Passenger

- PassengerID (Primary Key)
- Name

Bus

- BusID (Primary Key)
- Route

Booking

- BookingID (Primary Key)
- PassengerID (Foreign Key)
- BusID (Foreign Key)
- TravelDate

Relationship

- One passenger can make multiple bookings.
- One bus can have multiple bookings.

Viva Questions

Q1. Why should booking details be stored separately from passenger information?

Booking details are stored separately because a passenger can travel many times on different buses and dates. This design avoids data redundancy, keeps passenger information separate from travel history, and follows normalization principles.

Q2. If a booking contains multiple passengers, how would you redesign the schema?

Create a new table called **BookingPassenger**.

BookingPassenger

- BookingID (Foreign Key)
- PassengerID (Foreign Key)

This bridge table allows one booking to include multiple passengers while maintaining a normalized database structure.